PSEUDO CODE AND FLOWCHARTS

BY; MATLAB GROUP

THE NEWTON RAPHSON METHOD

- The Newton-Raphson method is an iterative numerical technique used to find approximate roots of an equation:
- $f(x)=0$
- The general Newton-Raphson formula is:
- $x_{n+1}=x_n-f(x_n)/f'(x_n)$
- where:
  - x_n= current approximation
  - x_{n+1}= next approximation
  - $f(x_n)$ = value of the function
  - $f'(x_n)$ = derivative of the function

Use Newton Raphson method to solve $f(x)=1-3x+1/2xe^{x}$

- $f(x) = 1 - 3x + \frac{1}{2}xe^{x}$
- $f'(x) = -3 + 1/2\, e^{x}(1 + x)$
- $x_{n+1} = x_n - (1 - 3x_n + 1/2x_n e^{xn})\ /(-3 + \frac{1}{2}\ e^{x}(1 + x)\ )$
- f(0)=1 , x_o=0.5 The root is in the region between 0 and 0.5
- $f(x) = 1 - 3(0.5) + \frac{1}{2}(0.5)e^{0.5}$
- $f(0.5) = -0.08782$
- $f'(0.5) = -3 + 1/2\, e^{0.5}(1 + 0.5)$
- $f'(0.5) = -1.76345$
- $x_2 = 0.45020033 - (0.002497)/(-1.8625887)$
- $x_2 = 0.4515409$
- $x_3 = 0.4515409 - (0.000001728)/(-1.86000996)$
- $x_3 = 0.45154190$
- $x_3 - x_2 = 0.000000929$
- $0.000000929 < 000001$
- SO, $x = 0.451419$
-

THE NRM PSEUDO CODE

- START
- ↓
- Define f(x) and f'(x)
- ↓
- Set $x_0 = 0.5$ and tolerance = 10^{-6}
- ↓
- Calculate xnew
- ↓
- Calculate error
- ↓
- Is error < tolerance?
- YES → Display root → STOP
- NO → Set x = xnew → Repeat

Using The “for” loop

```
for i = 1:maxIter
    fx = f(x);
    dfx = df(x);
    x_new = x - fx/dfx;
    error = abs(x_new-x);

    if error < tol
        break
    end

    x = x_new;
end
```

using the ‘while’ loop

- while error >= tol && i < maxIter
- i = i + 1;
- x_new = x - f(x)/df(x);
- error = abs(x_new-x);
- x = x_new;
- end

NRM FLOW CHART

START

↓

Define f(x) and f'(x)

↓

Set x_0, tolerance,
and maximum iterations

↓

Calculate
f(x), f'(x)

↓

xnew = x - f(x)/f'(x)

↓

error = |xnew - x|

↓

error <
tol?

YES | NO

N0.2; **SOLVE f(x)=x-2sinx=0 USING THE SECANT RULE**

- From $x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$
- $f(x) = x - 2sinx$
- Initial values $x_o = 1,\ x_1 = 2$
- Tolerance: 10^{-6}

SECANT ITERATION AND RESULTS

Iteration	Xn−1	Xn	Xn+1
0	1.000000	2.000000	1.790125
1	2.000000	1.790125	1.889121
2	1.790125	1.889121	1.895915
3	1.889121	1.895915	1.895493
4	1.895915	1.895493	1.895494
5	1.895493	1.895494	1.895494

- Therefore;

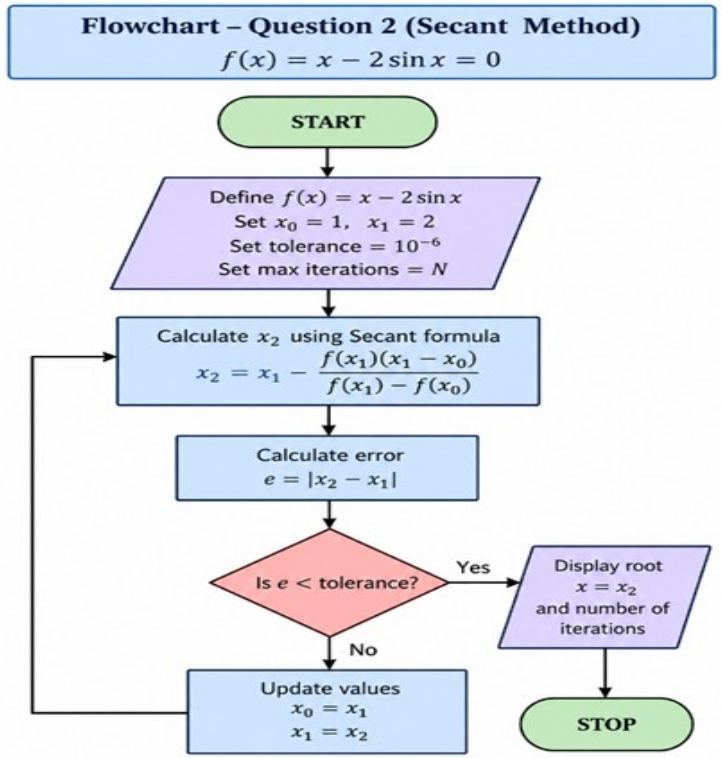
$$x = 1.895494$$

Secant pseudo code and flowchart

Pseudo code

- START
- ↓
- Define f(x) = x - 2sin(x)
- ↓
- Set $x_0 = 1$, $x_1 = 2$
- Set tolerance = 10^{-6}
- ↓
- Calculate x_2 using Secant formula
- ↓
- Calculate error = $|x_2 - x_1|$
- ↓
- Is error < tolerance?
- YES → Display root → STOP
- NO → $x_0 = x_1$
- $x_1 = x_2$
- Repeat

flowchart



The “for” loop

```
f = @(x) x - 2*sin(x);

x0 = 1;
x1 = 2;
tol = 1e-6;
maxIter = 100;

for i = 1:maxIter
    x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0));
    error = abs(x2-x1);

    if error < tol
        break
    end

    x0 = x1;
    x1 = x2;
end

fprintf('Root = %.8f\n',x2);
```

The “while” loop

```
f = @(x) x - 2*sin(x);

x0 = 1;
x1 = 2;
tol = 1e-6;
error = Inf;
i = 0;

while error >= tol
    i = i + 1;

    x2 = x1 - f(x1)*(x1-x0)/(f(x1)-f(x0));
    error = abs(x2-x1);

    x0 = x1;
    x1 = x2;
end

fprintf('Root = %.8f\n',x1);
```

NO.3

Use fourth order Runge Kutta method to solve the algorithm with step size h=0.2 to solve the initial value problem $\frac{dy}{dx} + 2y = sin3x$ with y(0)=1

- With $y(0) = 1$
- And h=0.2,
- $0 \leq x \leq 2.4$
- $\frac{dy}{dx} + 2y = sin3x$
- Therefore $f(x, y) = sin3x - 2y$
- $k_1 = hf(x_n, y_n)$
- $k_2 = hf(x_n + \frac{h}{2}, y_n + \frac{k_1}{2})$
- $k_3 = hf(x_n + \frac{h}{2}, y_n + \frac{k_2}{2})$
- $k_2 = hf(x_n + h, y_n + k_3)$
- $y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + 2k_4)$

But h=0.2

x	y
0.0	1.000000
0.2	0.721532
0.4	0.612924
0.6	0.573054
0.8	0.522623
1.0	0.416748
1.2	0.250514
1.4	0.053902
1.6	-0.123237
1.8	-0.231648
2.0	-0.241923
2.2	-0.156146
2.4	**-0.008087**

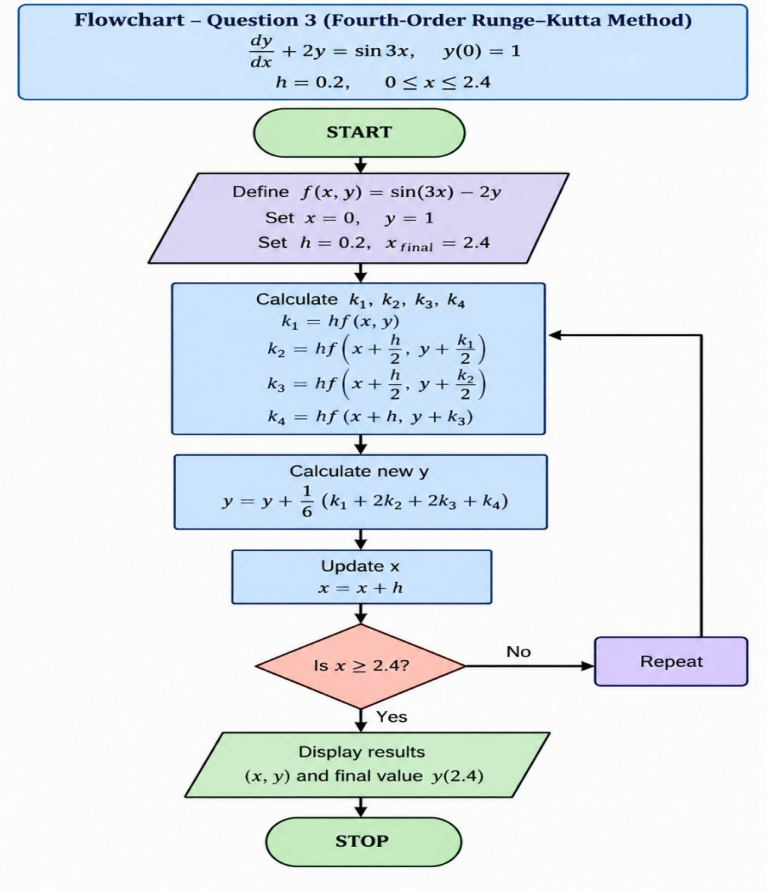
- So, $y(2.4) \approx 0.008087$

RK4 PSEUDO CODE /FLOWCHART

PSEUDO CODE

- START
- ↓
- Define f(x,y) = sin(3x) - 2y
- ↓
- Set x = 0, y = 1
- Set h = 0.2
- Set final x = 2.4
- ↓
- Calculate k1
- Calculate k2
- Calculate k3
- Calculate k4
- ↓
- Calculate new y
- ↓
- Update x
- ↓
- Is x ≥ 2.4?
  - YES → Display results → STOP
  - NO → Repeat

FLOWCHART



The ‘for’ loop

```
h = 0.2;
x = 0;
y = 1;

n = round((2.4-0)/h);

for i = 1:n

   f = @(x,y) sin(3*x) - 2*y;

   k1 = h*f(x,y);
   k2 = h*f(x+h/2,y+k1/2);
   k3 = h*f(x+h/2,y+k2/2);
   k4 = h*f(x+h,y+k3);

   y = y + (k1+2*k2+2*k3+k4)/6;
   x = x + h;

   fprintf('x = %.1f, y = %.6f\n',x,y);
end
```

The ‘while’ loop

```
h = 0.2;
x = 0;
y = 1;

while x < 2.4

    f = @(x,y) sin(3*x) - 2*y;

    k1 = h*f(x,y);
    k2 = h*f(x+h/2,y+k1/2);
    k3 = h*f(x+h/2,y+k2/2);
    k4 = h*f(x+h,y+k3);

    y = y + (k1+2*k2+2*k3+k4)/6;
    x = x + h;

    fprintf('x = %.1f, y = %.6f\n',x,y);
end
```